

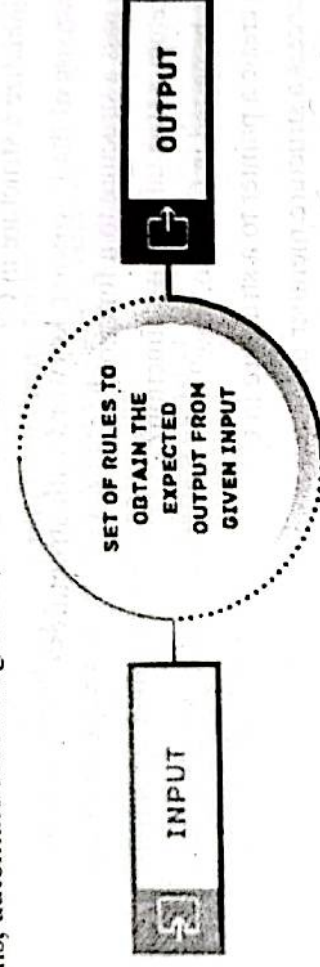
CHAPTER – 6

ALGORITHMS AND FLOWCHARTS

ALGORITHMS

6.1 Algorithm

An algorithm is a step-by-step procedure or set of rules designed to perform a specific task or solve a particular problem. It is a logical sequence of actions that leads to the desired outcome or solution. Algorithms are essential in computer science and programming, where they are used to manipulate data, perform calculations, automate reasoning tasks, and much more.



6.1 Algorithm

Key Characteristics of an Algorithm:

1. **Initianness:** An algorithm must always terminate after a finite number of steps.
2. **Definiteness:** Each step of the algorithm must be precisely defined and unambiguous.
3. **Input:** An algorithm has zero or more inputs, which are the data provided before the algorithm begins.
4. **Output:** An algorithm has one or more outputs, which are the results of the algorithm's computations.
5. **Effectiveness:** Each step of the algorithm must be simple enough that it can, in principle, be carried out by a person using only pencil and paper.

Example of an Algorithm:

Problem: Find the largest number in a list of numbers.

Algorithm:

1. Start with the first number in the list and assume it is the largest.
2. Compare the assumed largest number with the next number in the list.
3. If the next number is larger than the current largest number, update the largest number.
4. Repeat steps 2-3 for all the numbers in the list.
5. After completing the comparisons, the current largest number is the largest number in the list.
6. End.

Types of Algorithms:

- **Sorting Algorithms:** Examples include Bubble Sort, Merge Sort, Quick Sort.
- **Search Algorithms:** Examples include Linear Search, Binary Search.

Chapter – 6: Algorithms and Flowcharts

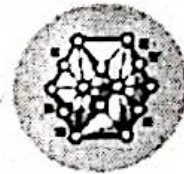
- **Recursive Algorithms:** Algorithms that call themselves to solve smaller instances of the same problem.
- **Dynamic Programming:** An optimization technique where the problem is broken down into overlapping subproblems.
- **Greedy Algorithms:** Algorithms that make locally optimal choices at each step with the hope of finding a global optimum.



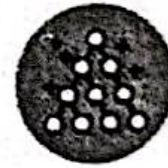
Brute Force
Algorithm



Divide and
Conquer
Algorithm



Dynamic
Programming
Algorithm



Greedy
Approach



Backtracking
Algorithm

6.2 Types of Algorithm

Use in Programming:

Algorithms are implemented as code in programming languages to automate tasks, solve problems, or perform computations. They are fundamental to software development, data processing, artificial intelligence, and many other fields in computer science.

Understanding algorithms and their design is crucial for efficient problem-solving and optimizing the performance of software applications.

Linear Search - Find 'G' in sorted list A-R

6.13 Binary Search and Linear Search

6.11 Difference between Linear Search and Binary Search

Aspect	Linear Search	Binary Search
Definition	A search algorithm that checks each element of a list sequentially until the target value is found or the end of the list is reached.	A search algorithm that repeatedly divides a sorted list in half to locate a target value efficiently.
Input Requirement	Can be used on unsorted or sorted lists.	Requires a sorted list to function correctly.
Approach	Sequentially scans through each element from the beginning to the end of the list.	Divides the search interval in half repeatedly to narrow down the location of the target value.
Time Complexity	$O(n)$ - Linear time complexity, where n is the number of elements in the list.	$O(\log n)$ - Logarithmic time complexity, where n is the number of elements in the list.
Space Complexity	$O(1)$ - Constant space complexity, as it requires no additional storage beyond the input list.	$O(1)$ - Constant space complexity, assuming the binary search is implemented iteratively. ($O(\log n)$ if implemented recursively due to stack space.)
Performance	Generally slower for large lists due to the need to potentially examine every element.	Much faster for large lists due to the logarithmic reduction in the number of comparisons needed.
Use Case	Useful for small or unsorted lists where the overhead of sorting or additional requirements for binary search are not justified.	Ideal for large, sorted lists where efficiency in search time is critical.

the specific requirements of the application, such as the expected number of elements, the frequency of insertions/deletions, memory constraints, and the desired lookup speed.

FLOWCHARTS

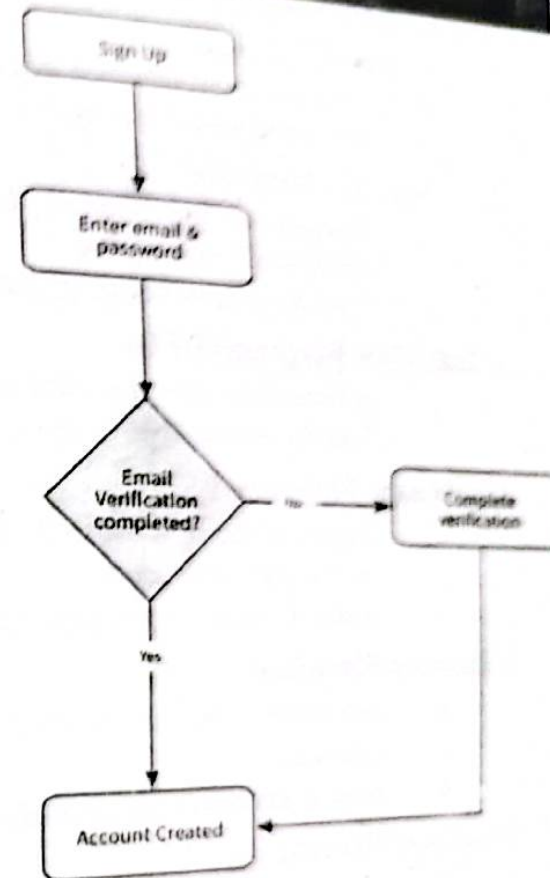
6.17 Introduction of Flowchart

A **flowchart** is a diagrammatic representation of a process or algorithm. It uses a set of standardized symbols and arrows to illustrate the steps involved in a process, decision-making paths, and the flow of control from one step to another. Flowcharts are widely used in various fields, including software engineering, business process modeling, education, and more, to visualize, document, and analyze processes.

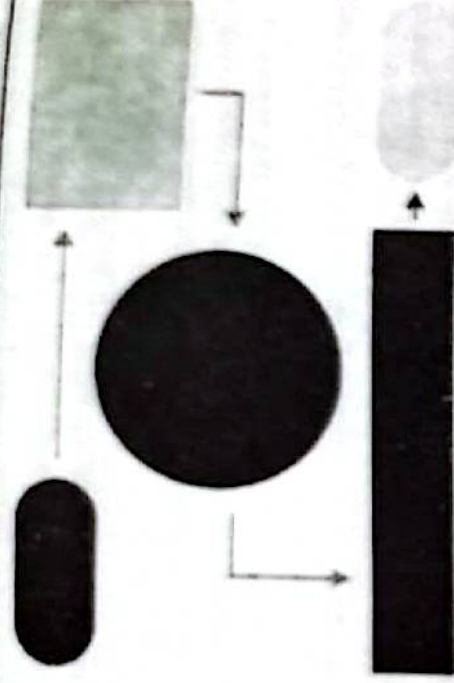
Key Components of a Flowchart

1. Flowchart Symbols:

- **Oval (Terminator):** Represents the start or end of a process.
- **Rectangle (Process/Operation):** Indicates a process, action, or operation to be performed.
- **Diamond (Decision):** Represents a decision point, where the flow can branch based on a yes/no question or true/false condition.
- **Parallelogram (Input/Output):** Used to show input to or output from a process, such as data entry or displaying information.
- **Arrows (Flow Lines):** Indicate the direction of flow from one step to the next.
- **Circle (Connector):** Used to connect different parts of the flowchart, often in complex diagrams where multiple flows converge or diverge.



6.1 Flowchart



6.2 Flowcharts Symbols

2. Flow Direction:

- Flowcharts typically flow from top to bottom or left to right, following the natural reading direction. Arrows indicate the direction of the flow.

3. Connecting Symbols:

- Arrows connect the symbols to show the sequence of operations and decisions, guiding the user through the process.

Types of Flowcharts:

1. Process Flowchart:

- Depicts the sequence of steps in a process or system.
- Commonly used in process documentation, quality control, and workflow analysis.

2. System Flowchart:

- Focuses on the flow of data within a system, illustrating how data moves between components or modules.
- Used in systems design and analysis, especially in software development.

3. Data Flow Diagram (DFD):

- A specialized flowchart that represents the flow of data within a system.
- Focuses on how data is input, processed, stored, and output.

4. Swimlane Flowchart:

- Organizes the flowchart into lanes, each representing a different department, role, or entity involved in the process.
- Useful in business process modeling to illustrate responsibilities and interactions.

5. Decision Flowchart:

- Emphasizes decision points within a process, with multiple branches based on different outcomes.
- Used in decision-making analysis and troubleshooting.

Benefits of Flowcharts:

- Visualization:** Flowcharts provide a clear visual representation of a process, making it easier to understand complex systems or procedures.
- Communication:** Flowcharts are an effective tool for communicating processes and workflows to stakeholders, including team members, clients, and management.
- Problem Solving:** By breaking down a process into individual steps, flowcharts help identify bottlenecks, inefficiencies, and areas for improvement.